

Injeção de Dependência

Curso básico de Go – Aula 05

Pré-requisitos: Aulas 01–04 (todas as anteriores)

0 problema

Hello retorna string – e daí?

```
func Hello(options ...HelloOption) string {  
    // ... monta saudação ...  
    return greeting + ", " + cfg.Name  
}  
  
func main() {  
    fmt.Println(Hello(WithName("Maria"))) // sempre stdout  
}
```

- ✗ E se eu quiser escrever num **arquivo**?
- ✗ E se eu quiser mandar pra uma **resposta HTTP**?
- ✗ E se eu quiser capturar num **buffer** pra testar?

Printf tem destino fixo no **stdout** – dependência **hard-coded**

Investigando o código-fonte

Printf é só um atalho

```
func Printf(format string, a ...interface{}) (n int, err error) {  
    return Fprintf(os.Stdout, format, a...)  
}
```

Printf = **Fprintf** com destino **fixo** em **os.Stdout**

Fprintf aceita qualquer destino

```
func Fprintf(w io.Writer, format string, a ...interface{}) (n int, err error)
```

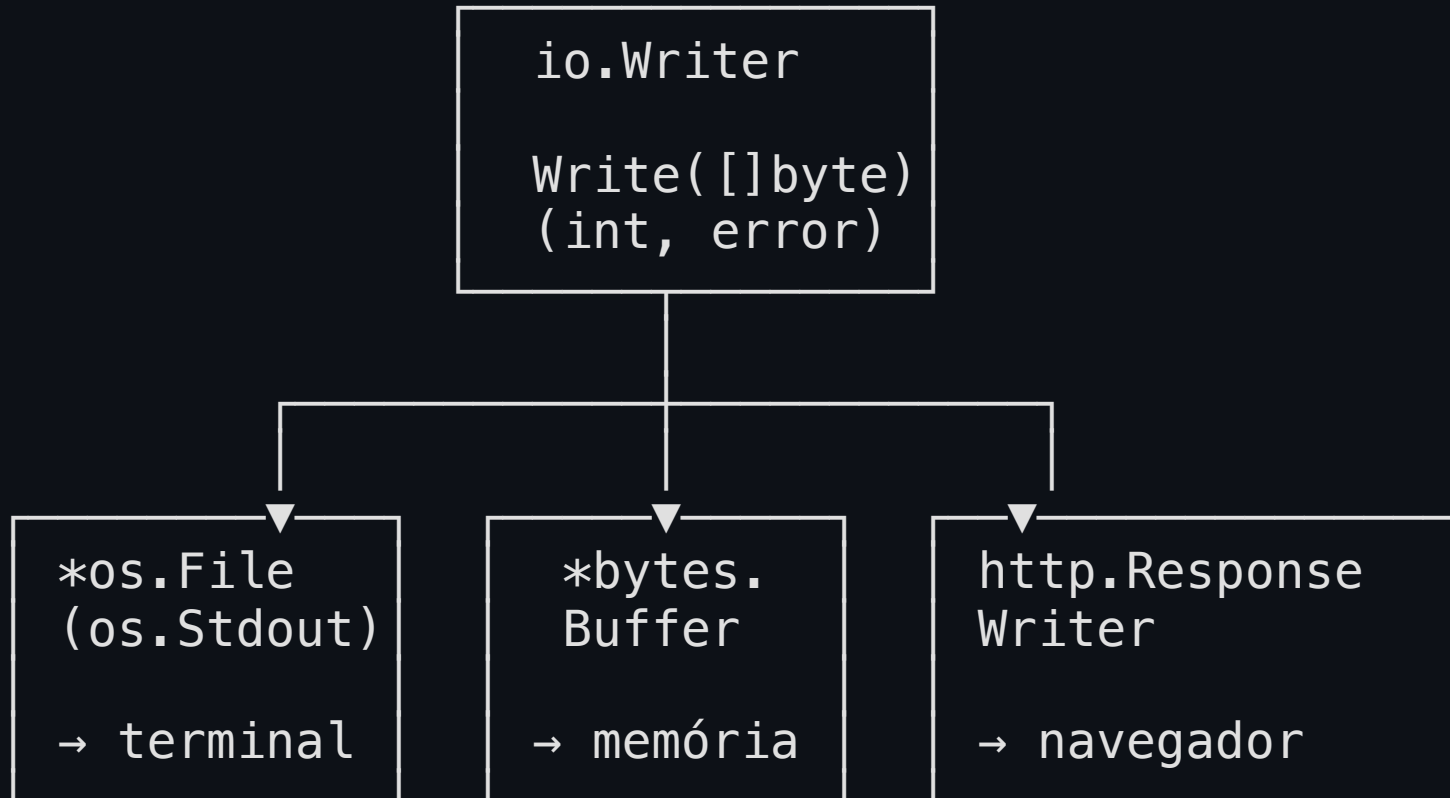
O primeiro parâmetro é **io.Writer** — uma **interface**

`io.Writer` – uma interface muito importante no Go

```
type Writer interface {  
    Write(p []byte) (n int, err error)  
}
```

Um único método. Qualquer tipo que o tenha é um `io.Writer`.

Quem implementa `io.Writer`?



Uma função, três contextos

Contexto	Escritor	O que acontece
Teste	<code>*bytes.Buffer</code>	Dados vão pro buffer → inspecionar
Terminal	<code>*os.File</code> (stdout)	Dados aparecem no terminal
HTTP	<code>http.ResponseWriter</code>	Dados vão pro navegador

```
// Teste
buffer := bytes.Buffer{}
Saudar(&buffer, WithName("Maria"))

// Terminal
Saudar(os.Stdout, WithName("João"))

// HTTP
func MeuHandler(w http.ResponseWriter, r *http.Request) {
    Saudar(w, WithName("Renata"))
}
```

Composição, não substituição

```
Hello(options...) → gera a saudação (retorna string)  
Saudar(escriptor, opts...) → escreve no destino (usa Hello por dentro)
```

Nenhum código anterior foi alterado. Só adicionamos uma camada.